
id: MDCS-TICKET-002

title: Implement schema compressor utility.

epic: epic-dumb-copilot

status: ready

risk: medium

allowed_areas:

- tools/schema_compressor.py

- tools/requirements.txt

must_not_touch:

- procedures/

- references/

requirements:

- Create `tools/schema_compressor.py`.

- Implement two extraction modes:

1. **DDL File Mode:** Parse a local `.sql` file containing DDL commands (`CREATE TABLE`, primary key constraints, foreign key constraints).

2. **Database Query Mode:** Connect to an Oracle database using the python `oracledb` (thin client) driver, and query system tables for schema information.

- Compress the schema into a compact text notation:

- * All uppercase table and column names.

- * Concise type mappings (e.g., `VARCHAR2(100)` -> `VARCHAR(100)`, `NUMBER(38,0)` -> `NUM`, `DATE` -> `DATE`).

- * Appended constraints in brackets: PK for primary keys, FK -> `TABLE.COL` for foreign keys.

- * Example output: `EMPLOYEES(employee_id NUM PK, first_name VARCHAR(50), department_id NUM FK -> DEPARTMENTS.department_id)`

- Save the output to a text file or print it to console.

non_goals:

- Do not parse complex PL/SQL packages, triggers, or views. Focus strictly on table definitions.

acceptance_criteria:

- Running `python tools/schema_compressor.py --file test_ddl.sql` outputs the correct compressed notation.

- Database queries gather primary keys and foreign keys correctly.

verification_commands:

- `python tools/schema_compressor.py -h`

- `python -m unittest discover -s tools/ -p "test.py"` (if tests are written)

depends_on:

- MDCS-TICKET-001

Body

Goal

Create a python command-line tool that translates verbose Oracle DDL or database metadata into an extremely dense notation for prompt injection.

Context

Copy-pasting standard `CREATE TABLE` scripts drains the limited context window of GPT-3.5. A compact representation of column names, shortened types, and key relationships saves up to 90% of the token cost while preserving the information necessary to write joins and filters.

Procedure

1. Add `oracledb` to `tools/requirements.txt`.
2. Write `tools/schema_compressor.py` with standard library `argparse`.
3. Implement DDL parsing using robust regex groups (capturing column definitions and key constraints).
4. Implement Oracle connection using the thin driver. Write queries to fetch table structure:
 - Columns: `select TABLE_NAME, COLUMN_NAME, DATA_TYPE, DATA_LENGTH, DATA_PRECISION, DATA_SCALE from USER_TAB_COLUMNS`
 - Constraints: `select CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, R_CONSTRAINT_NAME from USER_CONSTRAINTS`

- Columns in constraints: `select CONSTRAINT_NAME, TABLE_NAME, COLUMN_NAME, POSITION from USER_CONS_COLUMNS`
5. Map references and output the compressed string.